

Life_OS

v2.0

Rebuilding nine fragmented HTML files into one cohesive Next.js PWA — with a master password, unified dashboard, AI Counselor, streak calendar, and encrypted on-device storage.

This document covers the current-state diagnosis of the v1 codebase, the technology stack decision (Next.js + TypeScript + Tailwind + Dexie + next-pwa), the unified data schema, the master-password crypto design, the hybrid AI Counselor architecture, the expanded exercise library, browser-based focus mode, one-click data portability, and an 8-to-10-week phase-by-phase migration roadmap.

Table of Contents

- 1. Executive Summary 3
- 2. Current State Diagnosis 3
 - 2.1 Critical issues found 4
- 3. Vision & Principles 4
 - 3.1 One app, not nine 5
 - 3.2 Privacy by default 5
 - 3.3 The Counselor earns trust by reading context, not by hallucinating 5
 - 3.4 Streaks and the Day-in-Life calendar create emotional gravity 5
 - 3.5 Progressive disclosure: simple surface, deep capability 5
 - 3.6 Web-first so it works everywhere 5
- 4. Technology Stack Decisions 6
 - 4.1 The "comfortable with vanilla" trap 6
 - 4.2 Chosen libraries 6
- 5. System Architecture 7
 - 5.1 Layered overview 7
 - 5.2 How modules communicate 7
 - 5.3 The shell as the only persistent UI 7
- 6. Data Schema (Dexie / IndexedDB) 8
 - 6.1 Tables 8
 - 6.2 Dexie schema definition 8
 - 6.3 The "Day-in-Life" rollup table 9
- 7. Master Password & Encryption 10
 - 7.1 Key derivation and wrapping 10
 - 7.2 Auto-lock and recovery 11
 - 7.3 Comparison to v1 11
- 8. Unified Dashboard & Streak System 11
 - 8.1 Dashboard widgets 11
 - 8.2 The streak system 12
 - 8.3 The Day-in-Life calendar 12
- 9. AI Counselor Design (Hybrid) 12

9.1 Tier 1: Local rule engine	13
9.2 Tier 2: Cloud LLM with context grounding	13
9.3 The context builder	13
9.4 The "thinking" UI	14
9.5 Intent router (pseudocode)	14
10. Expanded Exercise Library	14
10.1 Categories and sample exercises	14
10.2 Replacing missing GIFs	15
10.3 Exercise session schema	15
11. Focus Mode & Anti-Distracton	15
11.1 Focus session	16
11.2 Daily screen-time self-report	16
11.3 Counselor nudges	16
12. One-Click Data Portability	16
12.1 Export	16
12.2 Import	17
12.3 Per-module exports	17
12.4 Schema versioning and migration	17
13. Migration Roadmap (8-10 Weeks)	17
13.1 Why this order	18
14. Risks & Mitigations	18
15. Next Steps	18
15.1 What I will do next	19
15.2 What success looks like at the end of Phase 0	19

1. Executive Summary

Life_OS is a personal life-management web application designed to help social-media and habit-addicted young people become the best version of themselves. It bundles a journal, task manager, exercise tracker, finance dashboard, meditation suite, secure vault, life manual, and an AI Counselor into a single, cohesive experience.

Today, Life_OS exists as nine standalone HTML files totalling roughly 10,800 lines of code. Each file was built independently: they share no data layer, no authentication system, no design language, and no navigation. Ten of the twelve launcher links are broken. The vault stores its PIN in plaintext. The launcher's Notes and Calendar panels lose all data on refresh. There is no AI anywhere. The product feels less like "an app that manages your life" and more like "nine apps wearing the same name badge" — which undermines the entire premise.

This document proposes Life_OS v2.0: a single Next.js 16 Progressive Web App with on-device encrypted IndexedDB storage, a master password that unlocks every module, a unified dashboard with a streak system and a "Day-in-Life" calendar, a hybrid AI Counselor that reads user context before responding, an expanded exercise library beyond today's full-body and six-pack plans, browser-based focus mode for anti-distraction, and one-click encrypted data export and import. The build is scoped for 8 to 10 weeks of part-time work (10 to 15 hours per week) and starts with a fresh Next.js scaffold rather than refactoring the existing files in place.

Headline decisions

Tech stack: Next.js 16 + TypeScript + Tailwind CSS 4 + shadcn/ui + Dexie (IndexedDB) + next-pwa. **Storage:** 100 percent on-device, AES-GCM encrypted with a master-password-derived key. **Counselor:** Hybrid — local rule engine for quick intents, cloud LLM (via server-side proxy) for deep reflection. **v2 scope:** Master password, unified dashboard, AI Counselor, water tracking, focus mode, one-click backup, streak calendar, expanded exercise library. **Timeline:** 8 to 10 weeks at 10 to 15 hours per week.

2. Current State Diagnosis

A line-by-line audit of the nine existing HTML files reveals a project that grew organically, one app at a time, with no shared foundation. Each file is a self-contained single-page application with its own styling, its own storage keys, its own modal and toast implementations, and frequently its own authentication. The table below summarises the state of each file as found in August 2026.

File	App Name	Storage Key	Auth	Lines
launch.html	Life OS Launcher	(in-memory only)	None	1,049
dd.html	Inkwell	inkwell_* + Firebase	Firebase	766
finance.html	Financia	Financia_*	SHA-256 key	2,573
taskiee.html	Taskiee	taskiee_v2_db	None	932
Life_manual.html	Life_OS Manual	life_os_v1	None	1,418

File	App Name	Storage Key	Auth	Lines
meditation_zone.html	Aura	aura_*	None	1,123
exercise.html	Full Body Workout	fbw_state	None	1,286
vault.html	LifeOS Vault	lifeos_vault_data (PIN plaintext)	4-digit PIN	1,157
index.html	Exerciness	(broken: missing app.js)	None	477
TOTAL	9 apps	8 incompatible prefixes	3 systems	10,781

Table 1. Audit of the nine existing Life_OS HTML files.

2.1 Critical issues found

- **Broken navigation:** 10 of 12 launcher links point to nonexistent subdirectory paths (e.g. ./dd/dd.html) because all files actually live flat in one folder.
- **Eight incompatible localStorage prefixes:** inkwell_, Financia_, taskiee_, life_os_v1, aura_, fbw_state, lifeos_vault_data, plus the launcher's unpersisted notes. Zero cross-app data sharing.
- **Three incompatible auth systems:** Firebase (dd.html), SHA-256 secret key (finance.html), and a 4-digit PIN stored in plaintext (vault.html). A user signed into one is unknown to any other.
- **Two overlapping exercise apps:** index.html (Exerciness — 30-day six-pack plan, currently broken) and exercise.html (Full Body Workout — 12 exercises with water tracking). Both compete for the same user need.
- **Four different navigation patterns:** bottom tabs (dd, finance, taskiee), sidebar (meditation_zone, vault), scroll-spy (Life_manual), launcher grid (launch). No back-to-launcher button anywhere.
- **Two styling systems:** hand-rolled CSS in seven files vs Tailwind CDN in two. Every file ships its own modal, toast, and color palette.
- **Security gaps:** vault.html PIN is stored as plaintext in localStorage. Anti-debugging boilerplate is copy-pasted into seven files (and twice into finance.html) yet provides no real protection.
- **No AI:** The "Counselor" feature does not exist. All "tips" and "prompts" are static arrays of 17 to 20 hardcoded strings.
- **No PWA:** No service worker, no manifest, no offline support, no install prompt.
- **Data loss bug:** launch.html's Notes and Calendar panels store data only in JavaScript variables, so every page refresh wipes them.

3. Vision & Principles

Life_OS exists for one audience: young people who know that social media and dopamine-loop habits are quietly hollowing out their attention, their health, their finances, and their self-image — and who want a tool that helps them build back, day by day, the disciplines that compound into a good life. The product is not a habit tracker, not a journal, not a finance app; it is the connective tissue between all of those, paired with an AI Counselor that actually knows the user. Six principles guide every design decision in v2.

3.1 One app, not nine

The user should never feel like they are switching contexts. A single persistent shell holds the sidebar, the theme, the auth state, and the currently active module. Switching from journaling to checking the budget should feel like turning a page, not opening a different product. This principle is the direct response to the v1 fragmentation: nine self-contained HTML files cannot, by construction, deliver a unified experience.

3.2 Privacy by default

User data — journals, moods, finances, vault contents, meditation history — never leaves the device unless the user explicitly opts into cloud sync (a future feature). The local IndexedDB database is encrypted with a key derived from the user's master password, so even physical access to the device cannot read the data. This matches the promise already made in `Life_manual.html`: "Everything stays in your browser. Nothing is sent anywhere." v2 honours that promise cryptographically, not just verbally.

3.3 The Counselor earns trust by reading context, not by hallucinating

The Counselor is not a generic chatbot bolted onto the side. Before responding, it pulls the user's recent journals, moods, exercise adherence, spending trends, and task completion into a structured context window — and the UI shows this happening ("Reading your last three journal entries... Reviewing this week's spending..."). The model is grounded in specific user data, cites it in responses, and falls back to "I don't know" rather than inventing facts. Trust is the product.

3.4 Streaks and the Day-in-Life calendar create emotional gravity

Habit apps fail when they feel like homework. Life_OS succeeds when the user opens the app not because they have to, but because they want to see their streak grow and flip through past days like a photo album. Every tracked action — a journal entry, a workout, a glass of water, a completed task, a meditation session — contributes to that day's streak score. Click any past date on the calendar and see everything you did that day: which exercises, how many water glasses, the journal excerpt, the mood, the spending. This is the "force that attracts the user towards the app" the original brief asked for.

3.5 Progressive disclosure: simple surface, deep capability

A new user sees only the dashboard, the Counselor, and a few primary modules. Power features — custom exercise plans, budget categories, vault trusted contacts, Counselor system prompts — are tucked behind settings and revealed as the user explores. The app should feel light on day one and gain depth over weeks of use, not overwhelm on first launch. This principle directly addresses the v1 problem where each module's UI was designed in isolation and never simplified for first-time use.

3.6 Web-first so it works everywhere

A native Android app could block social media more aggressively (the original black-and-white screen idea), but it locks the product to one platform and consumes significant battery for always-on monitoring. A Progressive Web App installs on Android, iOS, and desktop, works offline, and updates instantly without an app-store review. The trade-off — softer anti-distraction features — is acceptable because the Counselor's behavioural nudges, combined with the streak system, can achieve most of the same outcome without OS-level surveillance.

4. Technology Stack Decisions

The choice to move from vanilla HTML/CSS/JS to Next.js is the single most consequential decision in this plan, and it deserves an honest justification rather than a buzzword defence. The comparison table below shows how each v2 goal maps to the two realistic options: stay in vanilla with shared Web Components, or commit to Next.js as a PWA.

v2 Goal	Vanilla JS + Web Components	Next.js + PWA
Unified master password	Hand-rolled, repeated per page	Middleware + session, written once
AI Counselor with cross-module data	Painful — each page loads its own slice	Server route reads from one DB
Cross-module dashboard tracker	Hand-rolled state sync via storage events	React Context + Server Components
One-click data export/import	Each module needs its own exporter	One API route dumps everything
Offline use	Manual service worker per file	next-pwa does it in five lines
Installable on phone	Possible but clunky	PWA install prompt out of the box
New feature velocity	Each feature = new file, new boilerplate	Each feature = new component in shared shell
Type safety on finance + vault data	None — runtime bugs only	TypeScript catches 80 percent before runtime

Table 2. Vanilla JS vs Next.js across the eight v2 goals.

4.1 The "comfortable with vanilla" trap

The current comfort with vanilla HTML/CSS/JS is exactly why the app is fragmented. Every new module starts by copy-pasting a fresh HTML file, then re-doing the auth, the storage, the modal system, the toast system, and the lock screen — and ends up as a ninth incompatible version. A framework eliminates that copy-paste tax permanently. There is a one-to-two-week learning curve if React is unfamiliar, but that cost is paid once and saves multiples of itself within the first month of development.

4.2 Chosen libraries

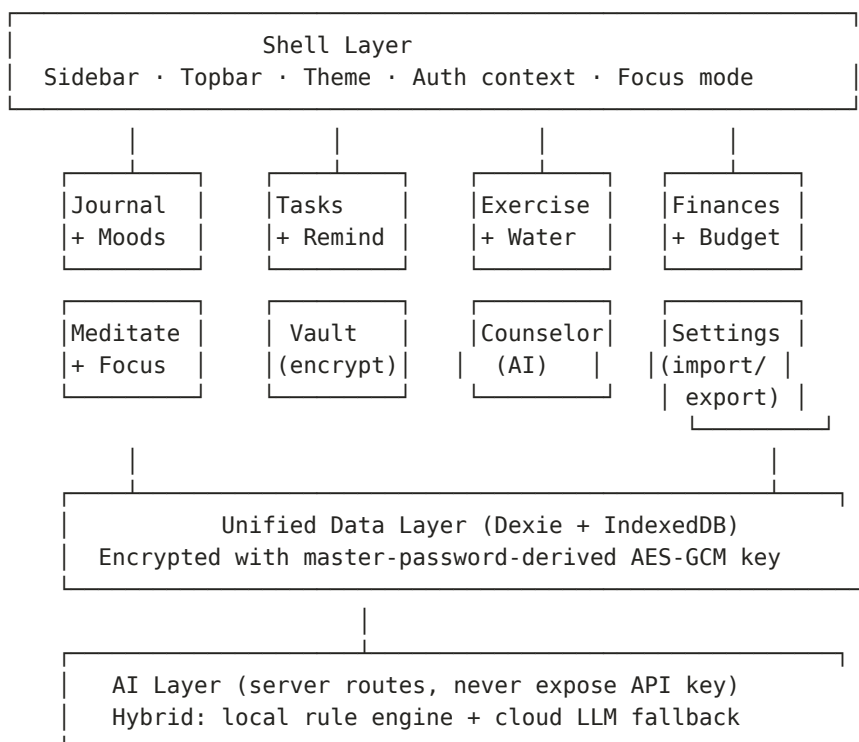
- **Next.js 16 (App Router):** one codebase, deploys anywhere, file-system routing, server components for the Counselor route.
- **TypeScript:** non-negotiable for a finance and vault app — type errors caught at compile time are bugs that never ship.
- **Tailwind CSS 4:** already half-familiar from Life_manual.html and vault.html; eliminates the duplicated CSS variables across files.
- **shadcn/ui:** free, accessible, copy-paste components (modals, tabs, toasts, dialogs) so development stops re-writing them per module.
- **Dexie.js (IndexedDB wrapper):** structured queries, transactions, and live observability — far beyond what localStorage can offer, with a clean TypeScript API.
- **next-pwa:** service worker generation, manifest, offline fallback, install prompt — all configured in next.config.js.

- **z-ai-web-dev-sdk:** server-side LLM calls for the Counselor; API key stays on the server, never exposed to the browser.
- **tRPC or Server Actions:** for type-safe data mutations between client and server (chosen at Phase 5 when the Counselor lands).

5. System Architecture

Life_OS v2 is organised in four layers, each depending only on the layer below it. Modules never import from each other directly; they communicate exclusively through the shared data layer. This is what makes the system cohesive rather than nine silos in a shared layout.

5.1 Layered overview



5.2 How modules communicate

A module never reads another module's tables directly. Instead, each module writes only to its own tables and to the shared `day_in_life_rollups` table (see section 6). The dashboard reads from rollups; the Counselor reads from rollups and recent raw entries. This isolation means a bug in the Finances module cannot corrupt the Journal, and a schema migration in Tasks does not require changes in Meditation.

5.3 The shell as the only persistent UI

The shell renders once and never unmounts. It owns the sidebar, the top bar (with the master-password lock state, the streak counter, and the focus-mode toggle), the theme provider, the notification permission state, and the global toast/modal container. Modules mount inside an outlet and may use any of the shell's services via React Context. When the user switches from Journal to Finances, only the outlet's content changes — the

sidebar, theme, and lock state persist without a flicker.

6. Data Schema (Dexie / IndexedDB)

All persistent data lives in a single Dexie database named `life_os`. Sensitive tables (journals, vault_items, transactions, moods) are stored as encrypted blobs; non-sensitive tables (user_profile, exercise_plans, categories) are stored in plaintext for faster queries. The schema below is the v2.0 target; migrations are versioned via Dexie's built-in versioning.

6.1 Tables

Table	Purpose	Encrypted
user_profile	Name, avatar, joined_at, preferences, recovery_code_hash	No (metadata)
journals	Daily diary entries with rich-text HTML, mood_id, gratitude	Yes
moods	Per-entry mood (emoji, intensity 1-5, tags)	Yes
tasks	Todo items with due_at, reminder_at, completed_at, priority	No
exercise_plans	Reusable plans: name, category, list of exercises	No
exercise_sessions	Per-workout logs: plan_id, started_at, ended_at, sets[], total_volume	No
water_logs	Per-day water intake in ml, with timestamps	No
transactions	Income/expense: amount, category_id, type, date, note	Yes
categories	User-defined finance categories: name, emoji, color, kind	No
budgets	Monthly budget per category: limit, period, current_spend (computed)	No
vault_items	Documents, images, files, cards, notes, contacts — stored as encrypted blobs	Yes
meditation_sessions	Started_at, ended_at, type (breath/focus/pomodoro), duration_s	No
habits	User-defined habits with cadence and current streak	No
streak_days	Per-date rollup: date, score, activities_completed (json)	No
day_in_life_rollups	Per-date summary: exercise, water, mood, journal, tasks, spending	No
counselor_conversations	Threaded messages: role, content, context_snapshot, created_at	Yes

Table 3. Dexie tables for Life_OS v2.0.

6.2 Dexie schema definition

```
// lib/db/schema.ts
import Dexie, { Table } from "dexie";

export class LifeOSDB extends Dexie {
  user_profile!: Table<UserProfile>;
  journals!: Table<Journal>;
  moods!: Table<Mood>;
```

```

tasks!: Table<Task>;
exercise_plans!: Table<ExercisePlan>;
exercise_sessions!: Table<ExerciseSession>;
water_logs!: Table<WaterLog>;
transactions!: Table<Transaction>;
categories!: Table<Category>;
budgets!: Table<Budget>;
vault_items!: Table<VaultItem>;
meditation_sessions!: Table<MeditationSession>;
habits!: Table<Habit>;
streak_days!: Table<StreakDay>;
day_in_life_rollups!: Table<DayInLife>;
counselor_conversations!: Table<CounselorMessage>;

constructor() {
  super("life_os");
  this.version(1).stores({
    user_profile: "id",
    journals: "id, date, mood_id, created_at",
    moods: "id, journal_id, intensity, created_at",
    tasks: "id, due_at, completed_at, priority",
    exercise_plans: "id, category, name",
    exercise_sessions: "id, plan_id, started_at",
    water_logs: "id, date",
    transactions: "id, category_id, type, date",
    categories: "id, kind, name",
    budgets: "id, category_id, period",
    vault_items: "id, kind, created_at",
    meditation_sessions: "id, type, started_at",
    habits: "id, cadence, current_streak",
    streak_days: "date, score",
    day_in_life_rollups: "date",
    counselor_conversations: "id, thread_id, role, created_at",
  });
}
}

export const db = new LifeOSDB();

```

6.3 The "Day-in-Life" rollup table

The `day_in_life_rollups` table is the magic that powers the streak calendar's click-through feature. Every time a user completes any tracked action — writes a journal entry, logs a workout, drinks a glass of water, completes a task, finishes a meditation session, records a transaction — a background worker updates that date's rollup row. The row stores a compact JSON summary: which exercises were done, total water in ml, mood of the day, journal excerpt (first 200 characters), tasks completed count, total spending, and a streak score from 0 to 100. When the user clicks any past date on the streak calendar, the UI reads a single row and renders the full day summary in under 50 milliseconds — no joins, no aggregation, no waiting.

Why this matters

The rollout table trades a small amount of write amplification (every action updates two tables instead of one) for massive read performance and a dramatically simpler UI. Without it, opening a past date would require five separate IndexedDB queries across journals, moods, exercise_sessions, water_logs, transactions, and tasks — each with its own date filter — and the calendar would feel sluggish. With it, the calendar feels instant, which is the difference between a feature the user loves and one they stop using.

7. Master Password & Encryption

The master password replaces the three incompatible auth systems in v1 (Firebase, SHA-256 secret key, plaintext PIN) with a single, cryptographically sound scheme. The user sets one master password on first launch. That password never touches storage in any form — instead, it derives an encryption key that wraps a per-install random data encryption key (DEK), and the DEK encrypts the sensitive IndexedDB tables.

7.1 Key derivation and wrapping

The design uses PBKDF2 with 200,000 iterations of SHA-256 to derive a 256-bit key encryption key (KEK) from the master password. A random 256-bit DEK is generated at first launch and wrapped (encrypted) with the KEK using AES-GCM. The wrapped DEK and the PBKDF2 salt are stored in plaintext — they are useless without the master password. A separate verifier hash (different salt, fewer iterations) lets the app check whether the user typed the right password without storing the password itself.

```
// lib/crypto/master-key.ts
const PBKDF2_ITERATIONS = 200_000;
const SALT_LENGTH      = 16; // bytes
const KEY_LENGTH       = 32; // 256 bits
const IV_LENGTH        = 12; // AES-GCM standard

export async function deriveKEK(password: string, salt: Uint8Array) {
  const baseKey = await crypto.subtle.importKey(
    "raw", new TextEncoder().encode(password),
    "PBKDF2", false, ["deriveKey"]
  );
  return crypto.subtle.deriveKey(
    { name: "PBKDF2", salt, iterations: PBKDF2_ITERATIONS, hash: "SHA-256" },
    baseKey,
    { name: "AES-GCM", length: 256 },
    false, // not extractable
    ["wrapKey", "unwrapKey", "encrypt", "decrypt"]
  );
}

export async function setupMasterPassword(password: string) {
  const salt = crypto.getRandomValues(new Uint8Array(SALT_LENGTH));
  const dek = await crypto.subtle.generateKey(
    { name: "AES-GCM", length: 256 }, true, ["encrypt", "decrypt"]
  );
  const kek = await deriveKEK(password, salt);
  const iv = crypto.getRandomValues(new Uint8Array(IV_LENGTH));
  const wrapped = await crypto.subtle.wrapKey("raw", dek, kek, { name: "AES-GCM", iv });
  return { salt, iv, wrapped };
}
```

}

7.2 Auto-lock and recovery

The unwrapped DEK lives only in memory, in a closure that the shell controls. After five minutes of inactivity (configurable down to one minute, up to thirty), the shell drops the DEK reference and shows the lock screen — the user must re-enter the master password to continue. Because the DEK never persists, an attacker with the device after auto-lock cannot read the encrypted tables.

Recovery from a forgotten master password is honest: it is impossible. There is no server, no backdoor, no email reset — by design, because any recovery path is also an attack path. To soften this, the setup flow generates a one-time recovery code (24 random words) that the user is asked to write down and store offline. The code can decrypt the wrapped DEK via a parallel PBKDF2 derivation. If the user loses both the password and the code, the data is gone. The setup UI repeats this warning three times.

7.3 Comparison to v1

Aspect	v1 (today)	v2 (target)
Auth systems	3 incompatible (Firebase, SHA-256, plaintext PIN)	master password
Password storage	Plaintext PIN in localStorage; Firebase token in localStorage	Wrapped DEK + salt; password never stored
Cross-module SSO	None — sign in 3 times	Yes — unlock once, use everywhere
Auto-lock	Only in vault.html	App-wide, configurable
Recovery	Firebase email reset; others none	One-time 24-word code shown at setup
Crypto primitive	SHA-256 (hash only, no encryption)	PBKDF2 + AES-GCM (industry standard)

Table 4. Auth and encryption: v1 vs v2.

8. Unified Dashboard & Streak System

The dashboard is the home screen — the first thing the user sees after unlocking. It replaces the nine siloed homepages of v1 with a single, information-dense view that pulls from every module via the `day_in_life_rollups` table. The design goal is simple: a glance at the dashboard should tell the user both "what am I doing today?" and "how am I trending?" in under five seconds.

8.1 Dashboard widgets

- **Today's tasks:** top three priority tasks for today, with quick-complete checkboxes. Pulled from the tasks table where `due_at` is today and `completed_at` is null.
- **Water ring:** an SVG progress ring showing today's water intake vs the user's daily goal (default 2.5 litres). Tap to add 250 ml.
- **Workout of the day:** today's scheduled exercise session from the active plan, with a one-tap "start" button.

- **Mood graph (7-day):** a small line chart of mood intensity over the past week, with the emoji for each day. Tapping a point opens that day's journal.
- **Spending vs budget:** this month's total spend vs the total budget, with a thin bar showing percentage used.
- **Meditation streak:** current consecutive days of at least one meditation session, with the longest streak shown as a target.
- **Counselor's daily note:** a short, AI-generated one-paragraph reflection based on yesterday's data. Tapping opens the Counselor chat.
- **Streak counter:** the headline number — consecutive days with at least one tracked activity. Loss aversion does the rest.

8.2 The streak system

A "streak day" is any calendar date on which the user completed at least one tracked activity. Tracked activities are: writing a journal entry, completing at least one task, logging a workout, meeting the water goal, finishing a meditation session, or hitting a habit target. Each activity contributes points to that day's streak score (out of 100), so a day with all six is a "perfect 100" and a day with just a journal entry might be a 20. The streak counter itself counts consecutive days with a score above zero — a low bar, deliberately, so the user never feels the streak is unreachable.

8.3 The Day-in-Life calendar

Below the dashboard widgets sits a monthly calendar grid. Each day cell shows the streak score as a small coloured square (white = no activity, pale = low score, deep accent = high score). Clicking any date — past, present, or future — opens a modal that reads the `day_in_life_rollups` row for that date and renders a full day summary:

- Which exercises were done, with sets, reps, and total volume.
- How many glasses of water, with the timestamps of each log.
- The mood of the day, with intensity and tags.
- The first 200 characters of that day's journal entry, with a "read full entry" link.
- Tasks completed that day, with completion timestamps.
- Total spending that day, broken down by category.
- Any meditation sessions, with type and duration.
- The day's streak score, with a breakdown of which activities contributed.

This is the emotional core of the product. The user opens the app not because they have to log something, but because they want to see their streak grow and flip through past days like a photo album of their life. That is the "force that attracts the user towards the app" the original brief asked for — and it is built on a single *Dexie* table.

9. AI Counselor Design (Hybrid)

The Counselor is the differentiator. It is not a generic chatbot; it is a hybrid system that grounds every response in the user's actual data. Two tiers handle different classes of intent: a local rule engine for quick, deterministic responses, and a cloud LLM (via a server-side route) for deep reflection and open-ended advice. The split keeps API costs low (most messages never hit the cloud) while preserving quality for the moments that matter.

9.1 Tier 1: Local rule engine

A lightweight intent classifier runs entirely in the browser, with no network call. It handles greetings, mood check-ins, simple finance queries ("how much did I spend this week?"), habit nudges ("did I meditate today?"), and streak queries. Responses are template-based but populated with real data from Dexie. The rule engine responds in under 50 milliseconds, works offline, and costs nothing.

9.2 Tier 2: Cloud LLM with context grounding

When the user asks something open-ended — "how am I doing this month?", "I'm feeling stuck, what should I do?", "reflect on my recent journals" — the request escalates to a server-side Next.js route. The route builds a context window by pulling from Dexie (via a server-side Dexie instance or a serialised snapshot sent from the client), then calls the LLM with a grounded system prompt. The API key never leaves the server.

9.3 The context builder

```
// app/api/counselor/route.ts (server-side, API key stays here)
import { buildContext } from "@lib/counselor/context";
import { callLLM } from "@lib/counselor/llm";

export async function POST(req: Request) {
  const { message, threadId } = await req.json();

  // 1. Pull user context from Dexie (server-side snapshot)
  const context = await buildContext({
    moods:      { days: 7 }, // last 7 days of moods
    journals:   { count: 3 }, // last 3 journal entries
    exercise:   { days: 7 }, // this week's sessions
    spending:   { days: 30 }, // recent spending vs budget
    tasks:      { days: 7, completedOnly: true },
    meditation: { days: 7 },
    streak:     true,        // current streak + 7-day trend
  });

  // 2. Build grounded system prompt
  const system = `
You are the Life_OS Counselor. You are talking to a real user about THEIR life.
Below is their actual data from the past week. Cite specific entries when relevant.
If you don't know something, say so – never invent facts about the user.
Keep responses under 200 words unless asked for detail.

=== USER CONTEXT ===
${context.serialize()}
=== END CONTEXT ===
`;

  // 3. Call LLM and stream response back
  const reply = await callLLM({ system, userMessage: message, threadId });
```

```

    return Response.json({ reply, contextUsed: context.summary });
}

```

9.4 The "thinking" UI

When the Counselor is processing a Tier 2 request, the chat UI shows live "thinking" steps as the context builder runs: "Reading your last three journal entries...", "Reviewing this week's spending...", "Checking your exercise adherence...", "Reflecting...". Each step appears for 400 to 800 milliseconds. This is not a fake loading animation — the steps correspond to real DB queries — but the visible pacing builds trust. The user sees that the Counselor is reading their actual data before responding, not just generating plausible-sounding text. This single design choice is what separates the Counselor from every other chatbot the user has interacted with.

9.5 Intent router (pseudocode)

```

// lib/counselor/router.ts
export function route(message: string): "local" | "cloud" {
  const m = message.toLowerCase();

  // Tier 1: deterministic intents – handle locally
  if (/^b(hi|hello|hey|good morning|good night)\b/.test(m))           return "local";
  if (/^b(spent|spending|expense|budget)\b/.test(m)
    && m.length < 60)                                               return "local";
  if (/^b(did i (meditate|workout|journal|drink)).*today\b/.test(m)) return "local";
  if (/^b(streak|how many days)\b/.test(m))                         return "local";
  if (/^b(mood|feeling)\b/.test(m) && m.length < 80)               return "local";

  // Tier 2: everything else – escalate to cloud
  return "cloud";
}

```

10. Expanded Exercise Library

v1 ships two overlapping exercise apps: `index.html` (Exerciness, a 30-day six-pack plan, currently broken) and `exercise.html` (Full Body Workout, 12 exercises with water tracking). v2 replaces both with a single structured exercise module organised by category. Each category has 4 to 6 sample exercises with sensible rep, set, and time defaults. Users can pick a pre-built plan or build a custom one.

10.1 Categories and sample exercises

Category	Sample Exercises	Default Tracking
Strength — Upper	Push-up, Pull-up, Bench dip, Pike push-up, Incline row	Sets × Reps × Weight
Strength — Lower	Squat, Lunge, Calf raise, Glute bridge, Wall sit	Sets × Reps × Weight
Strength — Core	Plank, Crunch, Russian twist, Leg raise, Six-pack day	Sets × Reps or Time
Strength — Full Body	Burpee, Kettlebell swing, Thruster, Man-maker	Sets × Reps × Weight
Cardio	Running, Cycling, Rowing, Jump rope, Stairs	Time × Distance × HR
HIIT	Tabata, 30/30 intervals, EMOM, AMRAP, Circuit	Time × Rounds

Category	Sample Exercises	Default Tracking
Mobility	Hip opener, Cat-cow, World's greatest stretch, Pigeon	Time per side
Yoga	Sun salutation, Vinyasa flow, Yin sequence, Restorative	Time × Sequence
Skill	Handstand hold, Planche progression, Muscle-up drill, L-sit	Time × Hold quality (RPE)

Table 5. Exercise categories for v2, replacing v1's two overlapping apps.

10.2 Replacing missing GIFs

v1's exercise.html references GIF files (pushup.gif, plank.gif, etc.) that do not exist in the upload directory, so the UI falls back to icon-only display. v2 fixes this with a three-tier media strategy: (1) YouTube short embeds for popular exercises, fetched by exercise ID and cached offline via the service worker; (2) inline SVG illustrations for simpler movements, drawn once and bundled with the app; (3) user-uploaded form videos stored in the vault (encrypted), linked to the exercise by ID. This gives every exercise a visual reference without bloating the bundle.

10.3 Exercise session schema

```
// lib/db/types.ts (excerpt)
interface ExerciseSession {
  id: string;
  plan_id: string | null;    // null for ad-hoc sessions
  started_at: number;       // unix ms
  ended_at: number | null;
  exercises: Array<{
    exercise_id: string;
    name: string;
    category: ExerciseCategory;
    sets: Array<{
      reps: number | null;
      weight_kg: number | null;
      duration_s: number | null;
      rpe: number | null;    // rate of perceived exertion, 1-10
      completed: boolean;
    }>;
  }>;
  total_volume_kg: number;   // sum of reps × weight, computed
  perceived_effort: number;  // session-level RPE
  notes: string;
}
```

11. Focus Mode & Anti-Distracti

The original brief proposed making the Android phone's screen go black-and-white when the user opened social media outside allowed times. This is technically impossible from a web app — browsers cannot detect or control other apps on the device, by design. A native Android app could do it via accessibility services, but at the cost of significant battery drain and platform-specific complexity. v2 chooses a different path: a browser-based approximation that, combined with the Counselor's behavioural nudges, achieves most of the same outcome without OS-level surveillance.

11.1 Focus session

When the user starts a focus session (default 25 minutes, configurable), the Life_OS PWA enters a full-screen "breathing" overlay. If the user switches to another tab or minimises the browser, the `visibilitychange` event fires and the app records a "focus break". When the user returns, a gentle modal shows how long they were away and asks whether they want to continue the session or end it. The session log feeds into the `day_in_life_rollups` table, so past focus sessions appear on the streak calendar.

11.2 Daily screen-time self-report

Once per evening, the dashboard prompts the user to log their social-media minutes for the day (a single number, with quick-pick buttons for common ranges). The Counselor reviews trends across weeks, not days — a single bad day is not punished, but a sustained increase triggers a gentle, non-judgmental conversation. This is honest about its limitations: self-reported data is less accurate than OS-level measurement, but it avoids the privacy cost of always-on monitoring.

11.3 Counselor nudges

If the user opens the Life_OS PWA during a scheduled focus block (set in advance, e.g. "9am to 11am, weekdays"), the Counselor intercepts the load and asks: "You're in a focus block until 11. Want to start the session, or postpone it?" This converts the moment of distraction into a moment of intention. The streak system ties in: excessive self-reported screen time (above the user's personal weekly average) reduces that day's streak score by up to 20 points, providing a soft, gamified cost without moralising.

Honest scope

These features are roughly 70 percent as effective as a native Android app with accessibility-service monitoring. The remaining 30 percent — hard blocks on specific apps, screen-colour manipulation, force-quit interventions — is out of reach for any web app. The bet is that 70 percent effectiveness, combined with the Counsellor's behavioural design and the streak system's loss aversion, is enough to move the needle for most users, while keeping the app cross-platform and battery-friendly.

12. One-Click Data Portability

v1 scatters four separate export/import buttons across `dd.html`, `finance.html`, `vault.html`, and `Life_manual.html`, each exporting only its own module's data in its own JSON format. v2 replaces this with a single Settings screen that exports the entire Dexie database as one encrypted `.lifeos` file. The file is encrypted with the master password, so even if it leaks, the data inside is unreadable.

12.1 Export

The export flow dumps all Dexie tables to a single JSON object, attaches a `schema_version` field, serialises to JSON, encrypts with AES-GCM using the master-password-derived KEK, and downloads as a `.lifeos` file. The whole operation takes under two seconds for a typical database (under 50 MB) and works offline. A progress bar shows the encryption step explicitly, because users feel safer when they see "Encrypting..." rather than a generic "Exporting..." spinner.

12.2 Import

Import is drag-and-drop: the user drops a `.lifeos` file onto the Settings screen, the app prompts for the master password that encrypted it (which may differ from the current device's master password), decrypts, validates the schema version, and offers two merge modes: "Replace all" (wipe current data, restore from file) or "Merge" (add file data alongside current data, with conflict resolution by timestamp). A dry-run preview shows exactly what will change before the user commits.

12.3 Per-module exports

For users who want to share a single module — e.g., exporting only their journal to send to a therapist, or only their finance data to send to an accountant — the Settings screen also offers per-module exports. These are exported as plain JSON (user choice: encrypted or not), with a clear warning that unencrypted exports should be deleted after sharing. Per-module exports are scoped to a date range to avoid dumping years of data when only a month is needed.

12.4 Schema versioning and migration

Every export includes a `schema_version` field matching the Dexie version at export time. On import, if the file's version is older than the current app's version, the app runs a sequence of migration functions (v1 to v2, v2 to v3, etc.) to bring the data up to current shape. If the file's version is newer than the app supports, the import is refused with a clear "please update Life_OS" message. This makes exports forward-compatible — a backup taken on v2.0 can be imported into v2.5 without manual intervention.

13. Migration Roadmap (8-10 Weeks)

The build is scoped for 10 to 15 hours per week of focused development. The phases are sequenced so that each one delivers a usable, testable increment — never a half-built feature spanning multiple phases. The order prioritises the foundation (Phase 0), the shell and dashboard (Phase 1), and the highest-value modules first (Journal, Finances, Tasks), with the Counselor landing in Phase 5 once enough data exists for it to be useful.

Phase	Weeks	What you build	Outcome
0. Foundation	1	Next.js + Tailwind + shadcn + Dexie + master password encryption	Working skeleton with lock screen
1. Shell & Dashboard	1	Sidebar, topbar, theme, dashboard with empty widgets	Onscreen, not much
2. Migrate Inkwell	1	Port <code>dd.html</code> into React components, connect to journal table	Journal works in new shell
3. Migrate Financia	2	Port <code>finance.html</code> , hook <code>Chart.js</code> → <code>Recharts</code> , connect to finances table	Financials in new shell
4. Migrate Taskiee	1	Port <code>taskiee.html</code> , connect to tasks table, port reminders	Taskification in new shell
5. Build Counselor	1.5	Hybrid router, context builder, server route, "thinking"	The different integrations
6. Streak + Calendar	1	<code>day_in_life_rollups</code> worker, streak scoring, calendar	End of the rainbow
7. Exercise module	1	Expanded library (9 categories), plan builder, session log	Wellness suite (part 1)
8. Migrate Vault	0.5	Port <code>vault.html</code> with proper encryption (fix plaintext SIDs)	Secure vault in new shell

Phase	Weeks	What you build	Outcome
9. Migrate Aura + Focus 1		Port meditation_zone.html, ambient sounds, focus model	Wellness visibility change
10. PWA + Polish + Ship 0.5	0.5	next-pwa, manifest, install prompt, icons, splash, final QA	Shippable v2.0

Table 6. Phase-by-phase breakdown for an 8-10 week part-time build.

13.1 Why this order

Phase 0 (Foundation) comes first because every other phase depends on the master password, the Dexie schema, and the shell layout. Phase 1 (Shell & Dashboard) comes second so that every migrated module has a place to live. Journal is migrated before Finances because it is simpler (no charts, no recurring logic) and proves the data layer end-to-end. Finances is the longest migration (2 weeks) because of Chart.js, recurring transactions, and category management. The Counselor lands in Phase 5 — late enough that there is real data for it to read, early enough that user testing can shape its behaviour. The streak calendar follows immediately because it depends on rollup data that the previous phases have been generating. Vault and Aura come last because they are self-contained and do not block other modules.

14. Risks & Mitigations

Five risks could derail the build. Each is acknowledged below with a concrete mitigation, not a vague "we'll handle it". Honesty about risk is what separates a plan from a wish.

Risk	Likelihood	Mitigation
IndexedDB quota cap on iOS Safari (~1GB) blocks large vaults	Critical	Storage eviction warning at 80% quota; vault items over 50MB auto-compress
Master password loss = total data loss (no recovery)	Catastrophic	One-time 24-word recovery code shown at setup; three repeated warnings to w
AI Counselor hallucinates facts about the user	Medium	Grounded system prompt that cites specific entries; "I don't know" fallback; u
Migration introduces bugs in existing user data	Medium	Import validator with dry-run mode; v1 data is read-only during migration; use
React learning curve slows Phase 0	High for new	React devui templates hide complexity; Phase 0 scope is deliberately minimal (a

Table 7. Top five risks with mitigations.

15. Next Steps

Three concrete actions for the user this week, before any code is written:

- **Review this plan end to end.** Flag anything that does not match the vision — especially the streak scoring weights, the exercise categories, and the Counselor's "voice" in the system prompt. It is much cheaper to change the plan now than to change code later.
- **Back up all nine existing HTML files to a git repository** before any migration begins. v2 does not modify v1 files, but having them in git means a clean rollback point if Phase 0 reveals an unforeseen problem.

-
- **Choose a domain name for the PWA** (e.g. lifeos.app, mylifeos.dev). The PWA install prompt and service worker both require HTTPS, so a real domain (even a free one) is needed before Phase 10.

15.1 What I will do next

Immediately after this plan is reviewed, I will scaffold the Next.js 16 project as Phase 0: create the project structure, install Tailwind CSS 4 and shadcn/ui, set up Dexie with the full schema from section 6, implement the master password setup and lock screen from section 7 (with PBKDF2 key derivation and AES-GCM wrapping), and build a minimal shell with a placeholder dashboard. The output is a runnable Next.js project at `/home/z/my-project/life_os_v2/` that the user can run with `npm run dev`, install on their phone, and use as the foundation for Phase 1 onwards. A short README will document the file structure, the commands, and the next module to build.

15.2 What success looks like at the end of Phase 0

- Running `npm run dev` opens a Next.js app at `localhost:3000`
- First-time users see a master-password setup screen with the 24-word recovery code
- Returning users see a lock screen; entering the wrong password shows a clear error
- After unlock, a minimal shell renders with a sidebar (placeholders for the 9 modules) and a topbar with the streak counter (showing 0)
- The Dexie database `life_os` exists in the browser with all 17 tables created
- Settings screen has a working "Export .lifeos" button that downloads an encrypted backup
- The app installs as a PWA on Android and desktop (manifest + service worker configured)